

Language Models, n-gram Context, and WikiText-103

A tutorial handout for undergraduate students with a statistics background

Prepared for UG tutorial discussion

June 2026

1 Learning goals

By the end of this tutorial, you should be able to:

1. Explain what a language model is in probabilistic terms.
2. Compute simple unigram, bigram, and trigram probabilities from counts.
3. Explain why smoothing is needed and why Kneser-Ney smoothing is important.
4. Define perplexity and interpret it as a test-set evaluation metric.
5. Distinguish between traditional n-gram models and neural language models.
6. Understand how to train and test language models on WikiText-103.
7. Design a fair experiment for comparing models with a fixed context length.

This handout assumes that you know basic probability, conditional probability, maximum likelihood estimation, logarithms, and the idea of splitting data into training and test sets. It does not assume prior knowledge of natural language processing.

2 What is a language model?

A language model is a probability distribution over sequences of tokens. A token could be a word, a punctuation mark, a subword piece, or a character. In this tutorial we mainly use word-level models, where a sentence is treated as a sequence of words:

$$w_1, w_2, \dots, w_T.$$

The central probability is

$$P(w_1, w_2, \dots, w_T).$$

Using the chain rule of probability, we can always write this as

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}).$$

This equation is the starting point for almost all language modeling. The task is to predict the next token from the previous tokens.

2.1 Example

Suppose the sequence is

`students enjoy probability`

A language model assigns

$$P(\text{students})P(\text{enjoy} \mid \text{students})P(\text{probability} \mid \text{students enjoy}).$$

The model is good if it gives high probability to fluent and meaningful text and low probability to unlikely text.

3 Why context length matters

The exact chain rule uses the entire previous history. For long documents, this history can be hundreds or thousands of words. A fixed-context n-gram model makes a simplifying assumption:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

That is, the model looks only at the previous $n - 1$ words when predicting the next word.

Terminology can be slightly confusing:

- A unigram model uses no previous words.
- A bigram model uses one previous word.
- A trigram model uses two previous words.
- An n-gram model usually uses $n - 1$ previous words to predict the current word.

For example, in a trigram model,

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-2}, w_{t-1}).$$

If the previous words are

`IIT Madras students enjoy`

then a trigram model predicting the next word after `enjoy` sees only

`students enjoy`

not the full earlier context.

4 Estimating n-gram probabilities from counts

Consider a bigram model. The maximum likelihood estimate is

$$\hat{P}(w_t = b \mid w_{t-1} = a) = \frac{C(a, b)}{C(a)},$$

where $C(a, b)$ is the number of times the bigram (a, b) appears in the training corpus and $C(a)$ is the number of times word a appears as a previous word.

For a trigram model,

$$\hat{P}(c \mid a, b) = \frac{C(a, b, c)}{C(a, b)}.$$

4.1 Small example

Suppose the training corpus is:

```
<s> I like probability </s>
<s> I like coding </s>
<s> students like probability </s>
```

Here `<s>` and `</s>` are start and end sentence markers.

The bigram probability of `like` after `I` is

$$\hat{P}(\text{like} \mid \text{I}) = \frac{C(\text{I like})}{C(\text{I})} = \frac{2}{2} = 1.$$

The bigram probability of `probability` after `like` is

$$\hat{P}(\text{probability} \mid \text{like}) = \frac{C(\text{like probability})}{C(\text{like})} = \frac{2}{3}.$$

This looks simple, but there is a serious problem.

5 The sparse data problem

Natural language has many possible word combinations. Even a corpus with millions of words will not contain every possible bigram, trigram, or 5-gram.

If a test sentence contains a trigram never seen in training, the maximum likelihood estimate gives it probability zero. Then the whole sentence gets probability zero.

That is too harsh. A sequence can be reasonable even if it was not seen before.

This motivates smoothing: we adjust the probability estimates so that **unseen or rare n-grams receive some probability mass**, while frequent n-grams still receive high probability.

6 Smoothing and backoff

A simple smoothing idea is **add-one smoothing**:

$$\hat{P}_{\text{add-one}}(w \mid h) = \frac{C(h, w) + 1}{C(h) + V},$$



where h is the history and V is the vocabulary size.

Add-one smoothing is easy to understand, but it is usually not competitive for language modeling because it gives too much probability to unseen events.

A better family of methods is backoff or interpolation. The idea is:

- Use the higher-order n-gram estimate when it is reliable.
- If the high-order context is rare or unseen, rely more on a lower-order model.

For example, a trigram model may combine trigram, bigram, and unigram evidence:

$$P(w_t | w_{t-2}, w_{t-1}) \approx \lambda_3 P_3(w_t | w_{t-2}, w_{t-1}) + \lambda_2 P_2(w_t | w_{t-1}) + \lambda_1 P_1(w_t).$$

The weights $\lambda_1, \lambda_2, \lambda_3$ must be chosen so that probabilities remain valid.

7 Kneser-Ney smoothing: the key idea

Kneser-Ney smoothing is one of the most important smoothing methods for n-gram language models. It is widely used because it gives strong performance in practice.

To understand its intuition, compare two words:

Francisco

and

the

The word **Francisco** may occur many times in a corpus, but it usually occurs after **San**. It is not a generally likely next word in many contexts. In contrast, **the** occurs after many different previous words.

A simple unigram model would think **Francisco** is common if it appears many times. Kneser-Ney asks a better question for lower-order probabilities:

In how many different contexts does this word appear?

For bigram Kneser-Ney, the lower-order probability of a word is based on the number of distinct preceding words it has, not just its raw count.

One common form of interpolated Kneser-Ney for bigrams is

$$P_{\text{KN}}(w | h) = \frac{\max(C(h, w) - D, 0)}{C(h)} + \lambda(h) P_{\text{cont}}(w),$$

DU, will understand this if required

where:

- h is the previous word or previous context.
- D is a discount, often between 0 and 1.
- The first term discounts observed counts.
- $\lambda(h)$ returns the leftover probability mass.
- $P_{\text{cont}}(w)$ is a continuation probability: how many different histories can precede w .

The continuation probability is

$$P_{\text{cont}}(w) = \frac{|\{h : C(h, w) > 0\}|}{|\{(h, w') : C(h, w') > 0\}|}.$$

This says: a word is likely as a continuation if it has appeared after many different histories.

7.1 Modified Kneser-Ney

Modified Kneser-Ney uses several discount values depending on whether an n-gram count is 1, 2, or at least 3. The details are more advanced, but the main message is simple:

Kneser-Ney is a carefully designed smoothing method that is especially strong for count-based n-gram language models.

For fixed-order n-gram baselines, modified interpolated Kneser-Ney is usually the default strong baseline.

8 Evaluating language models: cross-entropy and perplexity

Given a test sequence $w_1, \dots, w_T$, a language model assigns probabilities

$$P(w_t \mid w_{<t}).$$

The average negative log-likelihood per token is

$$H = -\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{<t}).$$

Remember, the goal is to MINIMIZE the negative log likelihood.

If the logarithm is natural, perplexity is

$$\text{PPL} = \exp(H).$$

If the logarithm is base 2, perplexity is

$$\text{PPL} = 2^H.$$

Both definitions are equivalent if the log base is handled consistently.

8.1 Interpretation

Perplexity can be interpreted as a kind of average branching factor. A perplexity of 100 means that, on average, the model is as uncertain as if it were choosing among about 100 equally likely words at each step.

Lower perplexity is better.

However, perplexity values are comparable only when the dataset, tokenization, vocabulary, and evaluation procedure are the same.

9 Why perplexity is not close to 1 on WikiText-103 [find what exactly is wikiText-103](#)

It is tempting to think that a very large neural model should get perplexity close to 1. On realistic word-level datasets such as WikiText-103, this is not the case.

A perplexity of 1 would mean the model assigns probability almost 1 to the correct next word almost every time. Natural language is too uncertain for that. Many next words can be plausible.

For example:

The students opened their ____

Many completions are possible: books, laptops, notebooks, assignments, and so on.

what exactly is
meant by
finite style
convolutional
models?

Strong published WikiText-103 results historically include perplexities around 37 for **finite-context convolutional models**, below 20 for stronger **Transformer-style models**, and around 18.3 for **Transformer-XL**. These numbers are far from 1 because the next-word prediction problem remains highly uncertain.

whats these?

10 What is WikiText-103?

WikiText-103 is a word-level language modeling benchmark built from high-quality Wikipedia articles. It is larger and more realistic than older small benchmarks such as the Penn Treebank.

Important properties:

- It contains over 100 million training tokens.
- It preserves case, punctuation, and numbers.
- It contains full articles, which makes long-range context potentially useful.
- It has a large vocabulary, roughly 267k word types in the commonly cited version.

The standard split has **train, validation, and test** files. The **validation set is used for model selection and hyperparameter tuning**. The test set should be used only for final reporting.

A common file layout is:

```
wikitext-103/  
  wiki.train.tokens  
  wiki.valid.tokens  
  wiki.test.tokens
```

11 Traditional n-gram models on WikiText-103

A simple experiment is to train n-gram models of different orders:

- $n = 1$: unigram
- $n = 2$: bigram
- $n = 3$: trigram
- $n = 4$: 4-gram
- $n = 5$: 5-gram
- higher orders if memory permits

For each n , train on `wiki.train.tokens`, tune any parameters on `wiki.valid.tokens`, and report perplexity on `wiki.test.tokens`.

11.1 Why higher n is not always automatically better

As n increases, the model has access to more context. This can help.

But the number of possible n -grams grows rapidly. Many high-order n -grams will be rare or unseen, so smoothing becomes more important.

Thus, performance depends on a tradeoff:

- Larger n gives more context.
- Larger n also increases sparsity.
- Smoothing determines how well the model handles sparsity.

11.2 Practical tool: KenLM

KenLM is a widely used toolkit for building and querying n -gram language models. It supports efficient storage and fast evaluation of large n -gram models.

A typical command-line workflow is:

```
# Train a 5-gram model with modified Kneser-Ney smoothing
lmplz -o 5 < wiki.train.tokens > wiki103_5gram.arpa
```

```
# Convert ARPA format to a compact binary format
build_binary wiki103_5gram.arpa wiki103_5gram.binary
```

```
# Evaluate perplexity on validation or test data
query wiki103_5gram.binary < wiki.valid.tokens
query wiki103_5gram.binary < wiki.test.tokens
```

The exact command names and options depend on the installation. For a serious experiment, record the KenLM version, pruning settings, vocabulary handling, and tokenization choices.

12 Neural language models

Traditional n -gram models store counts of observed word sequences. Neural language models instead learn numerical representations of words and contexts.

A neural word-level language model typically has:

1. An embedding vector for each word.
2. A context model that combines previous word embeddings.
3. A prediction layer that outputs a probability distribution over the vocabulary.

The model is trained to minimize negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P_{\theta}(w_t \mid \text{context}).$$

This is the same objective used to compute perplexity, but applied to the training data.

12.1 Why neural models can generalize better

Suppose a model has seen many examples involving words such as

student, professor, lecture, assignment, exam

A neural model can learn that these words have related meanings or usage patterns through their embeddings. It can then generalize better to combinations that did not occur exactly in training.

An n-gram count model cannot easily do this. If a particular trigram never occurred, it must rely on smoothing and lower-order counts.

13 Finite-context neural models

Your research question is especially interesting because it asks:

What is the best possible language model if the model is allowed to look back only n tokens?

This is different from asking for the best language model overall.

Several neural architectures can be made finite-context:

13.1 Feed-forward neural n-gram model

A feed-forward neural n-gram model takes the previous $n - 1$ word embeddings, concatenates them, passes them through one or more neural network layers, and predicts the next word.

For example, a 5-gram neural model predicts

$$P_{\theta}(w_t \mid w_{t-4}, w_{t-3}, w_{t-2}, w_{t-1}).$$

This is conceptually close to a traditional n-gram model, but it uses learned embeddings and neural network parameters instead of raw counts.

13.2 Convolutional language model

A convolutional language model applies filters over local windows of tokens. Stacking layers increases the receptive field, which is the number of previous tokens that can influence the prediction.

If the receptive field is 30 tokens, then the model cannot use information older than 30 tokens.

This makes convolutional models attractive for studying fixed-context language modeling.

Convolutional LM = Language Model built using CNNs

13.3 Transformer with limited attention window

A Transformer language model uses attention to combine information from previous tokens. If we restrict attention to the previous n tokens, it becomes a finite-context model.

This is not an n-gram count model, but it is an n-token context model.

14 Important distinction: n-gram order vs context window

Be careful with language.

A classical 5-gram model usually means:

$$P(w_t \mid w_{t-4}, w_{t-3}, w_{t-2}, w_{t-1}).$$

It uses 4 previous words.

A neural model with context length 5 may mean it uses 5 previous words to predict the next word. Authors are not always consistent.

When comparing papers, always ask:

1. Does n include the predicted word or only previous context?
2. Is the context exactly limited to n tokens?
3. Are there hidden states, recurrence, cache mechanisms, retrieval, or memory that leak longer context?
4. Is the evaluation word-level, subword-level, or character-level?
5. Is the vocabulary the same?

15 Training and testing on WikiText-103

A clean experimental pipeline is as follows.

15.1 Step 1: Download the data

Download the standard WikiText-103 release. Keep the original train, validation, and test split.

15.2 Step 2: Decide the tokenization

For classic WikiText-103 word-level evaluation, use the provided tokenized files directly.

Do not silently change tokenization. Lowercasing, removing punctuation, replacing rare words, or using subword tokenization changes the task.

15.3 Step 3: Build the vocabulary

For word-level models, the vocabulary is usually built from the training set. Words in validation or test that are not in training are mapped to an unknown token such as `<unk>`.

WikiText-103 has a low out-of-vocabulary rate compared with smaller datasets, but the vocabulary is still large.

15.4 Step 4: Train models

For n-gram models:

```
lmplz -o 1 < wiki.train.tokens > wiki103_1gram.arpa
lmplz -o 2 < wiki.train.tokens > wiki103_2gram.arpa
lmplz -o 3 < wiki.train.tokens > wiki103_3gram.arpa
```

```
lmlz -o 4 < wiki.train.tokens > wiki103_4gram.arpa
lmlz -o 5 < wiki.train.tokens > wiki103_5gram.arpa
```

For neural models:

- Choose architecture: feed-forward, CNN, RNN, Transformer.
- Fix the maximum context length.
- Train by minimizing cross-entropy on the training set.
- Use the validation set to choose learning rate, dropout, model size, and stopping point.

15.5 Step 5: Evaluate perplexity

Compute perplexity on validation during development. Report test perplexity once at the end.

The test perplexity is

$$\text{PPL}_{\text{test}} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid \text{allowed context}) \right).$$

15.6 Step 6: Make the comparison fair [need more explanation on this; will ask gpt](#)

For a fixed-context study, create a table like this:

Model	Allowed previous tokens	Tokenization	Validation PPL	Test PPL	Notes
Unigram KN	0	word			baseline
Bigram KN	1	word			modified KN
Trigram KN	2	word			modified KN
5-gram KN	4	word			modified KN
Neural 5-gram	4	word			feed-forward
CNN	30	word			finite receptive field
Transformer window	128	word/ subword			attention limited to 128

Do not mix word-level and subword-level perplexities unless you convert them carefully. A subword tokenization creates more tokens and changes the meaning of perplexity.

16 Suggested student exercise

16.1 Exercise 1: Count-based bigram model

Given this corpus:

```
<s> the cat sat </s>
<s> the dog sat </s>
<s> the cat slept </s>
```

1. Compute $P(\text{cat} \mid \text{the})$ using maximum likelihood. [2/3](#)
2. Compute $P(\text{dog} \mid \text{the})$. [1/3](#)
3. What probability does the model assign to $P(\text{mouse} \mid \text{the})$? [0/3 = 0](#)
4. Why is this a problem? [Because the model thinks that the pair "the mouse" can never exist. And that's why we need smoothing techniques.](#)

16.2 Exercise 2: Perplexity [cgpt](#)

Suppose a model assigns probabilities 0.5, 0.25, and 0.125 to the three correct next words in a test sequence.

1. Compute the average negative log-likelihood using natural logs.
2. Compute perplexity.
3. Interpret the result.

16.3 Exercise 3: Context limitation

For the sentence fragment:

After working through the proof carefully, the students finally understood the

↪ ----

Discuss what information is available to:

1. A bigram model. ["the"](#)
2. A 5-gram model. [students finally understood the](#)
3. A model with a 20-word context. [After working through the proof carefully, the students finally understood the](#)
4. A model with access to the whole paragraph. [After working through the proof carefully, the students finally understood the](#)

17 Research question for advanced students

The motivating research question is:

For each context length n , what is the lowest test perplexity achievable on WikiText-103 by any model that uses only the previous n tokens?

This is not fully answered by a single standard leaderboard. Most published leaderboards compare full architectures, not strict fixed-context limits.

A good project would be:

1. Train modified Kneser-Ney models for several n -gram orders.
2. Train neural feed-forward models with the same context sizes.
3. Train limited-window CNN or Transformer models.
4. Evaluate all models on the same WikiText-103 split.
5. Report perplexity, parameter count, training time, and memory use.

This project teaches an important scientific lesson: a fair comparison requires controlling the data, preprocessing, context length, and evaluation metric.

18 Reading guide

Start with the first three readings if you are new to the area.

18.1 Foundational language modeling and smoothing

1. 1. Chen, S. F. and Goodman, J. (1996/1998). *An Empirical Study of Smoothing Techniques for Language Modeling*. This is the classic empirical reference on smoothing methods for n-gram language models, including methods that led to strong Kneser-Ney baselines.
2. Kneser, R. and Ney, H. (1995). *Improved Backing-Off for M-Gram Language Modeling*. This is the original Kneser-Ney paper.
3. Goodman, J. (2001). *A Bit of Progress in Language Modeling*. This is a broad, readable discussion of many improvements to traditional language models.

18.2 Dataset

4. Merity, S., Xiong, C., Bradbury, J., and Socher, R. (2016). *Pointer Sentinel Mixture Models*. This paper introduced the WikiText language modeling benchmark, including WikiText-103.
5. Salesforce / Hugging Face WikiText dataset card. Useful for loading the dataset through modern Python tools.

18.3 Neural models and WikiText-103 results

2. 6. Bengio, Y., Ducharme, R., Vincent, P., and Jauvin, C. (2003). *A Neural Probabilistic Language Model*. A foundational paper for neural language modeling.
3. 7. Dauphin, Y. N., Fan, A., Auli, M., and Grangier, D. (2017). *Language Modeling with Gated Convolutional Networks*. Important for finite-context neural language modeling and WikiText-103.
8. Baevski, A. and Auli, M. (2018). *Adaptive Input Representations for Neural Language Modeling*. A strong Transformer-style result on WikiText-103.
9. Dai, Z. et al. (2019). *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. Important as a contrast: it improves performance by going beyond fixed-length context.

18.4 Tools

10. Heafield, K. (2011). *KenLM: Faster and Smaller Language Model Queries*. Practical toolkit for large n-gram language models.
11. Fairseq language modeling examples. Useful for reproducing convolutional and Transformer-style language model baselines.

19 References and useful links

The links below use short hyperlink text in the PDF so that the bibliography wraps cleanly within the page margins. The full URLs are stored as hyperlink targets in the LaTeX source.

- S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer Sentinel Mixture Models,” 2016. Introduces WikiText. arXiv:1609.07843.
- WikiText dataset summary by Stephen Merity. Includes dataset statistics. Stephen Merity dataset summary.

- Salesforce WikiText dataset card on Hugging Face. Hugging Face dataset card.
- S. F. Chen and J. Goodman, “An Empirical Study of Smoothing Techniques for Language Modeling,” ACL 1996 / Harvard TR 1998. ACL Anthology P96-1041.
- R. Kneser and H. Ney, “Improved Backing-Off for M-Gram Language Modeling,” ICASSP 1995. IEEE Xplore record.
- K. Heafield, “KenLM: Faster and Smaller Language Model Queries,” WMT 2011. ACL Anthology W11-2123.
- Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A Neural Probabilistic Language Model,” JMLR 2003. JMLR paper page.
- Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks,” ICML 2017. PMLR v70/dauphin17a.
- A. Baevski and M. Auli, “Adaptive Input Representations for Neural Language Modeling,” ICLR 2019. arXiv:1809.10853.
- Z. Dai et al., “Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context,” ACL 2019. ACL Anthology P19-1285.
- Fairseq language modeling examples. Fairseq language modeling examples.

20 Appendix: checklist for reporting results

When reporting a WikiText-103 language modeling result, include:

1. Dataset version and source.
2. Tokenization and vocabulary construction.
3. Whether evaluation is word-level, subword-level, or character-level.
4. Context limit in exact terms.
5. Model architecture.
6. Number of parameters.
7. Training hardware and approximate training time.
8. Validation perplexity.
9. Test perplexity.
10. Any pruning, caching, recurrence, retrieval, or memory mechanism.

Without these details, perplexity numbers may not be comparable.